

Linux 系统与基础操作

本文面向后续机器人开发所需的基础 Linux 使用，主要介绍 Ubuntu、Linux 文件系统、常用命令、Python 程序运行、Shell 脚本、权限、软件安装、环境变量、进程与设备文件等内容。后续使用 ROS2、MuJoCo/MjLab、强化学习框架以及进行 sim2real 和实机部署时，都会频繁使用这些知识。

1. Linux、Ubuntu 与终端

1.1 Linux 与 Ubuntu

严格来说，Linux 指 Linux 内核（Linux Kernel），Ubuntu、Debian、Arch Linux、Fedora 等是在 Linux 内核基础上构建的 Linux 发行版。机器人开发中通常直接将这类系统统称为 Linux 系统。

后续开发建议统一使用：

Ubuntu 22.04

统一系统版本可以减少 ROS2、Python、CUDA、仿真器以及各种软件依赖之间的版本冲突。

1.2 Terminal 与 Shell

Ubuntu 中可以使用 `Ctrl + Alt + T` 打开终端（Terminal）。终端主要负责命令的输入与输出，真正解析和执行命令的是 Shell，Ubuntu 中最常见的是 Bash。

例如：

```
source ~/.bashrc
```

这里的 `source`、路径和参数由 Bash 解析并执行。

因此，可以简单理解为：

Terminal：命令交互界面
Shell：命令解释器
Bash：一种常用 Shell

2. Linux 文件系统与路径

Linux 没有 Windows 中 `C:\`、`D:\` 这样的盘符结构，整个文件系统从根目录 `/` 开始组织。

常见目录包括：

目录	主要用途
<code>/home</code>	普通用户的个人文件
<code>/etc</code>	系统和软件配置文件
<code>/usr</code>	程序、库以及系统资源
<code>/opt</code>	第三方软件、SDK 等
<code>/dev</code>	系统中的设备文件
<code>/tmp</code>	临时文件

普通用户的主目录通常位于：

`/home/用户名`

可以使用 `~` 表示当前用户的主目录，例如：

`cd ~`

机器人开发中尤其需要关注 `/dev`。串口、USB 设备以及部分硬件接口通常会以设备文件形式出现，例如：

```
/dev/ttyUSB0  
/dev/ttyACM0  
/dev/ttyS0
```

2.1 绝对路径与相对路径

从根目录 `/` 开始书写的是绝对路径：

```
/home/user/robocon/project
```

相对于当前工作目录书写的是相对路径。例如当前目录为：

```
/home/user/robocon
```

则：

```
project
```

表示：

```
/home/user/robocon/project
```

三个常见特殊路径符号：

<code>.</code>	当前目录
<code>..</code>	上一级目录
<code>~</code>	当前用户主目录

例如：

```
cd ..  
cd ~  
./hello.sh
```

其中 `./hello.sh` 表示执行当前目录中的 `hello.sh`。

3. 常用文件与目录操作

3.1 查看目录

查看当前所在路径：

```
pwd
```

查看当前目录内容：

```
ls
```

常用参数：

```
ls -l      # 显示详细信息  
ls -a      # 显示隐藏文件  
ls -la     # 同时显示详细信息和隐藏文件
```

Linux 中以 `.` 开头的文件通常为隐藏文件，例如：

```
.bashrc  
.git
```

3.2 切换目录

```
cd dirname      # 进入子目录
cd ..           # 返回上一级
cd ~            # 返回用户主目录
cd /absolute/path
```

3.3 创建文件和目录

```
touch test.txt
mkdir demo
```

`touch` 常用于创建空文件，`mkdir` 用于创建目录。

3.4 复制、移动和重命名

复制文件：

```
cp file1 file2
```

复制目录：

```
cp -r dir1 dir2
```

移动文件：

```
mv file demo/
```

`mv` 同样可以用于重命名：

```
mv old_name.txt new_name.txt
```

3.5 删除

删除文件：

```
rm file.txt
```

删除目录：

```
rm -r directory
```

需要特别注意，命令行中的 `rm` 通常不会经过回收站，删除后很难恢复。使用：

```
rm -rf
```

前必须确认目标路径正确，尤其不要直接运行来源不明或自己无法理解的删除命令。

3.6 查看文本文件

对于较短文本，可以直接使用：

```
cat file.txt
```

实际开发中还会经常使用 `less`、`head`、`tail` 等工具，例如：

```
head file.txt  
tail file.txt  
tail -f log.txt
```

其中 `tail -f` 常用于实时观察程序日志。

4. 通配符与文件查找

Shell 支持使用通配符批量匹配文件，其中最常见的是 `*`，表示匹配任意数量的字符。

例如：

```
ls *.py
```

表示查看当前目录中的所有 Python 文件。

```
ls /dev/tty*
```

表示查看 `/dev` 下名称以 `tty` 开头的设备文件。

查找文件可以使用：

```
find . -name "*.py"
```

其中 `.` 表示从当前目录开始搜索。

5. Python 与程序运行

Ubuntu 中通常直接使用：

```
python3
```

查看 Python 版本：

```
python3 --version
```

运行 Python 文件：

```
python3 main.py
```

这条命令实际上包含两个部分：

<code>python3</code>	Python 解释器
<code>main.py</code>	需要执行的 Python 程序

系统首先寻找 `python3` 对应的程序，再由 Python 解释器读取并执行 `main.py`。

可以使用：

```
which python3
```

查看当前执行的程序位于什么位置，例如：

```
/usr/bin/python3
```

`which` 在使用 Python 虚拟环境、ROS2、CUDA 等环境时非常有用，因为同一个程序可能同时存在多个版本。

例如：

```
which python3
which pip
which code
```

VS Code

在已经配置 VS Code 命令行工具的情况下，可以使用：

```
code .
```


直接用 VS Code 打开当前目录。

机器人项目通常建议以“工程目录”为单位使用 VS Code，而不只是单独打开某一个源文件。

6. Shell 脚本与执行权限

当程序启动需要连续执行多条命令时，可以将命令保存到 Shell 脚本中，例如：

```
#!/bin/bash

source ~/.bashrc
cd ~/project
python3 main.py
```

通常将 Bash 脚本保存为：

```
xxx.sh
```

执行脚本可以使用：

```
./xxx.sh
```

如果出现：

```
Permission denied
```

通常说明文件没有执行权限。

Linux 文件权限

使用：

```
ls -l
```

可能看到：

```
-rw-r--r--
```

常见权限包括：

r	read, 读取
w	write, 写入
x	execute, 执行

为脚本增加执行权限：

```
chmod +x xxx.sh
```

之后即可：

```
./xxx.sh
```

实际项目中经常会看到：

```
chmod +x install.sh  
./install.sh
```

7. 软件安装与 CPU 架构

7.1 apt 软件包管理

Ubuntu 最常用的软件包管理工具之一是 `apt`。

更新软件包索引：

```
sudo apt update
```

安装软件：

```
sudo apt install tree
```

其中 `sudo` 表示以管理员权限执行当前命令。输入用户密码时，终端一般不会显示字符，这是正常现象。

`apt update` 主要更新本地的软件包信息，本身通常不会升级已经安装的软件。

7.2 .deb 软件包

部分软件会直接提供 `.deb` 安装包，可以使用：

```
sudo apt install ./package.deb
```

其中：

```
./
```

表示当前目录。

相比直接使用 `dpkg -i`，使用 `apt install ./xxx.deb` 通常能够更方便地自动处理依赖关系。

7.3 CPU 架构

下载 Linux 软件时经常会看到：

```
amd64  
x86_64  
arm64  
aarch64
```

它们表示 CPU 指令集架构。

查看当前系统架构：

```
uname -m
```

普通 Intel / AMD PC 通常输出：

```
x86_64
```

Jetson 等 ARM 平台通常输出：

```
aarch64
```

因此下载安装包时不仅要确认 Ubuntu 版本，还需要确认 CPU 架构是否匹配。

8. 环境变量与 .bashrc

环境变量用于给程序提供运行环境中的参数和路径信息，在 Python、CUDA、ROS2 和机器人 SDK 中非常常见。

创建环境变量：

```
export ROBOT_NAME=black
```

读取变量：

```
echo $ROBOT_NAME
```

输出：

```
black
```

通过 `export` 设置的变量通常只在当前 Shell 会话及其子进程中有效。关闭终端后重新打开，该变量一般不会继续存在。

如果希望每次打开 Bash 时自动设置某些环境变量，可以将命令加入：

```
~/ .bashrc
```

例如：

```
export ROBOT_NAME=black
```

修改后执行：

```
source ~/.bashrc
```

即可在当前 Shell 中立即重新加载配置，无需重新打开终端。

`source` 的作用可以理解为：在当前 Shell 环境中执行指定脚本中的命令。

后续 ROS2 中经常会遇到：

```
source /opt/ros/humble/setup.bash  
source install/setup.bash
```

本质上也是利用 `source` 修改当前终端的运行环境。

9. 进程基础

程序运行后会在 Linux 中形成进程（Process），每个进程都有对应的 PID（Process ID）。

查看当前终端相关进程：

```
ps
```

动态查看系统进程和资源占用：

```
top
```

如果知道某个进程 PID，可以发送终止信号：

```
kill PID
```

例如：

```
kill 12345
```

在终端中运行前台程序时，通常也可以使用：

```
Ctrl + C
```

终止当前程序。

对于机器人程序、ROS2 节点、训练程序和仿真器，进程管理是后续排查程序没有正常退出、GPU/CPU 被占用等问题的基础。

10. 串口与设备文件

Linux 将许多硬件设备抽象成文件，这些设备文件通常位于：

```
/dev
```

常见串口包括：

```
/dev/ttyS0
/dev/ttyUSB0
/dev/ttyACM0
```

一般可以初步理解为：

设备	常见含义
ttyS*	主机原生串口
ttyUSB*	USB 转串口设备
ttyACM*	USB CDC ACM 设备

机器人开发中的 USB 转串口模块、电机控制器、STM32、IMU 等设备都可能使用这些设备节点。

查找串口：

```
ls /dev/tty*
```

只查 USB 串口：

```
ls /dev/ttyUSB*
```

查找 ACM 设备：

```
ls /dev/ttyACM*
```

如果设备插入后不确定生成了哪个设备文件，可以在插入设备之后查看最近的内核日志：

```
dmesg | tail
```

后续实际使用串口时，还可能遇到设备访问权限问题，需要进一步理解用户组、`dialog` 权限以及 `udev` 规则。

11. 常用命令速查

当前目录

```
pwd
```

查看目录

```
ls
```

```
ls -l
```

```
ls -a
```

```
ls -la
```

切换目录

```
cd xxx
```

```
cd ..
```

```
cd ~
```

创建文件与目录

```
touch file
```



```
mkdir dir
```

```
# 复制
```

```
cp file1 file2
```

```
cp -r dir1 dir2
```

```
# 移动 / 重命名
```

```
mv old new
```

```
# 删除
```

```
rm file
```

```
rm -r dir
```

```
# 查看文本
```

```
cat file
```

```
head file
```

```
tail file
```

```
tail -f file
```

```
# 文件查找
```

```
find . -name "*.py"
```

```
# Python
```

```
python3 --version
```

```
python3 main.py
```

```
which python3
```

```
# Shell 脚本与权限
```

```
chmod +x script.sh
```

```
./script.sh
```

```
# 软件管理
```

```
sudo apt update
```

```
sudo apt install package
```

```
sudo apt install ./package.deb
```

```
# CPU 架构
```

```
uname -m
```

```
# 环境变量
export ROBOT_NAME=black
echo $ROBOT_NAME
source ~/.bashrc

# 串口与设备
ls /dev/tty*
ls /dev/ttyUSB*
ls /dev/ttyACM*
dmesg | tail

# 进程
ps
top
kill PID

# 其他
history
clear
```

12. 本阶段需要掌握的能力

完成这部分学习后，至少应能够独立完成以下操作：

- 理解 Linux、Ubuntu、Terminal 和 Shell 之间的基本关系；
- 理解 Linux 文件系统、绝对路径和相对路径；
- 使用命令行完成目录切换、文件创建、复制、移动、删除和查找；
- 使用终端运行 Python 程序，并通过 `which` 判断实际使用的程序；
- 编写并运行简单 Bash 脚本，理解基本文件权限；
- 使用 `apt` 和 `.deb` 安装软件，并区分 x86_64 与 aarch64；
- 理解环境变量、`.bashrc` 和 `source` 的作用；
- 完成基本的进程查看和终止；
- 在 `/dev` 中查找串口和其他硬件设备。

这些内容不要求一次记住全部命令。更重要的是理解 Linux 的基本组织方式，并能够在遇到实际问题时判断应该从路径、权限、环境、进程还是设备文件等方向进行检查。